

Képfeldolgozás – EAN-13 vonalkód feldolgozás

Makai Marcell - D53G1U

Tartalomjegyzék

1	BEVEZETÉS.....	3
2	A FELADAT ÉS A PROBLÉMA MEGHATÁROZÁSA.....	3
3	AZ EAN-13 KÓDRENDSZER ÁTTEKINTÉSE	3
3.1	AZ EAN-13 SZEREPE A TERMÉKAZONOSÍTÁSBAN	4
3.2	A 13 SZÁMJEGY FELÉPÍTÉSE	4
3.3	A 95 MODULBÓL ÁLLÓ VONALKÓDSZERKEZET	5
3.4	L, G, ÉS R KÓDOLÁSI MINTÁK	5
3.5	ELLENŐRZŐ SZÁMJEGY ÉS HIBAFELISMERÉS	5
3.6	QUIET ZONE ÉS A VONALKÓD KÖRNYEZETE	6
4	A PROGRAM FŐ MODULJAI	6
4.1	MAIN.PY	6
4.2	BARCODE_DETECTION.PY	6
4.3	IMAGE_PROCESSING.PY	6
4.4	SCANLINE.PY	7
4.5	PATTERN_DECODER.PY	7
4.6	DECODER.PY	7
4.7	EAN13_PATTERNS.PY	7
4.8	BARCODE_READER.PY	7
4.9	STATS_REPORT.PY	7
5	FELDOLGOZÁSI FOLYAMAT	7
6	ELŐFELDOLGOZÁS ÉS ROI DETEKTÁLÁS.....	8
7	SCANLINE-ALAPÚ DEKÓDOLÁS.....	8
8	RUN-ALAPÚ ÉS BIT-ALAPÚ DEKÓDOLÁS.....	8
9	HIBAKEZELÉS ÉS BIZONYTALANSÁGI LOGIKA.....	9
10	STATISZTIKA ÉRTÉKELÉSE	9
10.1	HIBAKATEGÓRIÁK ELEMZÉSE	9
10.2	SCORE ÉS TALÁLATSZÁM ÖSSZEHASONLÍTÁSA	9
10.3	A HIBÁS OLVASÁSOK JELENTŐSÉGE	9
10.4	MÓDSZERCSALÁDOK SZEREPE.....	9
10.5	ROI DETEKTÁLÁS ÉRTÉKELÉSE	9
11	EREDMÉNYEK ÉRTELMEZÉSE	9
12	FEJLESZTÉSI LEHETŐSÉGEK	9
13	ÖSSZEGZÉS.....	9
14	FORRÁSOK.....	9

1 Bevezetés

A projekt célja, egy Pythonban megvalósított, képfeldolgozáson alapuló EAN-13 vonalkódolvasó fejlesztése és tudásának értékelése [8]. Az alkalmazás feladata, hogy a termékfotókon automatikusan megtalálja a vonalkód-gyanús területeket, ezeket előfeldolgozza, majd scanline-alapú dekódolással megpróbálja kiolvasni az EAN-13 kódot belőle.

A legnagyobb kihívást az jelenti, hogy a vonalkód képek nem laborkörülmények között készültek, így gyakran ferdék, részben elmosódottak, eltérő megvilágításúak, csomagolásgörbületen helyezkednek el, vagy szöveges / táblázatos grafikai elemekkel keverednek. Ez miatt nem elég, ha dekódolni tudja az alkalmazás, hanem a bizonytalan vagy hibás olvasásokat is kezelnie kell.

A dokumentáció bemutatja az EAN-13 szabvány alapjait, az alkalmazás felépítését, az alkalmazott képfeldolgozási és dekódolási lépéseket, a hibakezelési logikát, valamint egy 1200 képből álló tesztadatbázison mért eredményeket. [1], [2], [7], [9]

2 A feladat és a probléma meghatározása

A megoldandó feladat egy olyan EAN-13 scanner elkészítése volt, amely képfájlokból képes automatikusan kiolvasni az azonosító számot. A bemeneti képek fájlneve tartalmazza az elvárt EAN-kódot, ezért a rendszer automatikusan össze tudja hasonlítani a dekódolt eredményt a várt értékekkel.

A feldolgozás eredménye három fő kategóriába sorolható:

- **OK:** a rendszer által kiolvasott EAN-13 kód megegyezik a fájlnevből származó elvárt értékkel.
- **WRONG / ROSSZ:** a rendszer adott vissza EAN-13 kódot, de az nem egyezik meg az elvárt értékkel.
- **ERROR / HIBA:** a rendszer nem adott vissza kódot, mert nem talált megfelelő ROI-t, nem tudta stabilan dekódolni a scanline-okat, vagy bizonytalanannak minősítette az eredményt.

A fejlesztés során a legfontosabb cél nem pusztán az olvasási arány növelése volt, hanem a hibás, de checksum-helyes hamis pozitív találatok csökkentése. Egy valós alkalmazásban a rossz vonalkód visszaadása súlyosabb probléma, mint az, ha a rendszer jelzi hogy nem biztos az olvasásban.

3 Az EAN-13 kódrendszer áttekintése

Az EAN-13 egy 13 számjegyből álló termékazonosító kódrendszer, amely a GS1 szabványrendszer részeként a kereskedelmi termékazonosítás egyik elterjedt formája [1],

[2]. A rendszer célja, hogy a termékeket egyértelműen azonosíthatóak legyenek a kereskedelmi, logisztikai és készletkezelési folyamatokban. A vonalkód grafikus formában jeleníti meg a számsorozatot, így optikai eszközökkel vagy képfeldolgozó algoritmusokkal automatikusan beolvasható.

A projekt szempontjából azért fontos az EAN-13 felépítésének ismerete, mert a program nem egyszerű karakterfelismerést végez, hanem a vonalkód szabványos sávstruktúráját próbálja visszafejteni. A sikeres dekódoláshoz a rendszernek fel kell ismernie a kezdő, középső és záró őrjeleket, a számjegyekhez tartozó sáv szélesség mintákat, a bal oldali paritás mintát és az ellenőrző számjegyet is [2], [8].

3.1 Az EAN-13 szerepe a termékazonosításban

Az EAN-13 kód a termékek globális azonosítására szolgál. A számsor általában egy GS1 vállalati azonosítóból, termékazonosító részből és egy ellenőrző számjegyből áll. A pontosan belső felosztás a kiadott azonosító hosszától és a GS1 szervezeti kiosztástól függ, de a gyakorlati felhasználás szempontjából a teljes 13 számjegy együtt azonosítja a terméket [1], [3].

A vonalkódolvasó alkalmazás számára ez azt jelenti, hogy a cél nem csupán egy tetszőleges 13 jegyű szám felismerése, hanem egy olyan számsor visszaadása, amely megfelel az EAN-13 formai és matematikai szabályainak. A dekódolás végén éppen ezért szükséges az ellenőrző számjegy vizsgálata. Ha a checksum nem egyezik, akkor a kapott kiolvasott számsor biztosan hibásnak tekinthető [2], [3].

3.2 A 13 számjegy felépítése

Az EAN-13 számsor 13 decimális számjegyből áll. Az első 12 számjegy hordozza az azonosító tényleges adattartalmát, a 13. számjegy az ellenőrző számjegy. Az ellenőrző számjegy számítása súlyozott összeggel történik: az egyes pozíciók értékei felváltva 1-es és 3-as súlyt kapnak, majd az eredmény alapján meghatározható az a számjegy, amely a teljes kódot tízzel oszthatóvá teszi [2], [3].

A programban ez a logika azért fontos, mert a dekódolt számjegysor csak akkor fogadható el jelöltként, ha az utolsó számjegy megegyezik a számított checksum értékével [8]. Ez azonban csak egy alapvető hibaszűrés. A checksum képes sok elírás vagy dekódolási hiba kiszűrésére, de nem bizonyítja teljes biztonsággal, hogy a kód valóban a képen szereplő vonalkódból származik.

Ez gyakorlatban azért lényeges, mert zajos vagy rosszul kivágott képrészleten is előfordulhat, hogy a rendszer véletlenül olyan 13 számjegyű sorozatot talál, amely a checksum szempontból érvényes. Emiatt a projektben a checksum mellett további megbízhatósági feltételek is szerepelnek, például a score, a találati szám, a módszercsaládok közötti egyezés és az alternatív jelöletek vizsgálata [8], [9].

3.3 A 95 modulból álló vonalkódszerkezet

Az EAN-13 grafikus vonalkódja nem közvetlenül a számjegyek képként történő felismerésén alapul, hanem egy szabványos modulrendszeren. A teljes EAN-13 vonalkód 95 azonos szélességű modulból áll [2]. Ezek a modulok fekete vagy fehér értéket vehetnek fel, és együtt alkotják a látható sávokat.

A 95 modul szerkezete:

- start guard: 3 modul
- bal oldali 6 számjegy: 6 x 7 modul
- middle guard: 5 modul
- jobb oldali 6 számjegy: 6 x 7 modul
- end guard: 3 modul

Ez összesen $3 + 42 + 5 + 42 + 3 = 95$ modul. A guard minták segítenek a vonalkód elejének, közepének és végének felismerésében. A start és end guard mintája fekete-fehér-fekete szerkezetű, a középső guard pedig elválasztja a bal és jobb oldali számjegycsoportot.

3.4 L, G, és R kódolási minták

Az EAN-13 számjegyei 7 modul szélességű mintákkal vannak kódolva. A jobb oldali hat számjegy R mintákkal jelenik meg, a bal oldali hat számjegy pedig L vagy G mintákkal. Az L és G minták nem véletlenszerűek: a bal oldali hat számjegy L/G sorrendje határozza meg az EAN-13 kód első számjegyét [2].

Ez azt jelenti, hogy az első számjegy nem külön 7 modulból álló vonalcsoportként szerepel a vonalkódon. Ehelyett a dekóder a bal oldali hat számjegy paritásmintájából következteti ki. Például, ha a bal oldalon minden számjegy L típusú mintával szerepel, az egy adott első számjegyet jelöl, más L/G kombinációk pedig más-más első számjegyhez tartoznak [2].

3.5 Ellenőrző számjegy és hibafelismerés

Az ellenőrző számjegy az EAN-13 egyik legfontosabb hibafelismerési eszköze. A dekódolt első 12 számjegyből a program kiszámítja a 13. számjegyet, majd összehasonlítja azt a ténylegesen olvasott utolsó számjeggyel [2], [3], [8]. Ha a két érték eltér, akkor a kód biztosan nem fogadható el.

Az ellenőrző számjegy ugyanakkor nem azonos a teljes olvasási bizonyossággal. A checksum csak a számsor matematikai helyességét vizsgálja, de nem ellenőrzi a képi eredet megbízhatóságát. Emiatt előfordulhat, hogy egy hibásan felismert, de véletlenül checksum-helyes számsor átmenne ezen az ellenőrzésen.

Emiatt a fejlesztett alkalmazásban további ellenőrzéseket csinálunk. A végső döntés több scanline eredményének összesítésén alapul. Ha ugyanazt a kódot sok scanline, több ROI-

forogtatás és több binarizálási módszer is támogatja, akkor az eredmény megbízhatónak tekinthető. Ha viszont csak kevés scanline ad azonos eredményt, vagy több checksum-helyes jelölt hasonló pontszámot kap, akkor a rendszer inkább bizonytalannak jelöli az olvasást [8], [9].

3.6 Quiet zone és a vonalkód környezete

Az EAN típusú vonalkódok olvasásánál fontos szerepet játszik a quiet zone, vagyis a vonalkód két oldalán található üres, zavaró mintáktól mentes terület. Ez segíti az olvasót abban, hogy pontosan felismerje, hol kezdődik és hol végződik a vonalkód [1], [2].

Termékfotókon ez gyakran problémát okoz. A vonalkód mellett közvetlenül szöveg, tápértéktáblázat, csomagolás grafika vagy más fekete-fehér elem helyezkedhet el. Ilyenkor a scanner egyszerűen tévesztheti össze a vonalkód sávjait a környező grafikai elemekkel, vagy a scanline nem tisztán a vonalkódon halad át.

4 A program fő moduljai

A program főbb, logikailag elkülönített modulból áll, a bemutatott moduláris felépítés a saját megvalósításom felépítését mutatja be [8]. Ez a felépítés átláthatóbbá teszi a fejlesztést és a dokumentálást, mert a képfeldolgozás, a ROI keresés, a scanline előállítás, a dekódolás és a statisztikai értékelés külön fájlokban található.

4.1 main.py

A fő belépési pont. Feladata a képek bejárása, az elvárt EAN-kód fájlnevből való kinyerése, a scanner meghívása, majd az eredmények kiírása. A futás során a program OK, ROSSZ vagy HIBA státuszt jelenít meg.

4.2 barcode_detection.py

A vonalkód-gyanús területek kereséséért felel. A megoldás éldetektálást, morfológiai műveleteket, kontúrkeresést és forogtatott téglalap alapú kivágást használ, amelynek az OpenCV képfeldolgozási könyvtára épülnek [4], [6]. A cél az, hogy a teljes képből olyan kisebb ROI-k készüljenek, amelyek nagy eséllyel tartalmazzák a vonalkódot.

4.3 image_processing.py

Az alapvető képfeldolgozási műveleteket tartalmazza: képbetöltés, szürkeárnyaltos konverzió, Otsu-küszöbölés és adaptív küszöbölés [5]. Ezek a lépések a változó fényviszonyok miatt szükségesek.

4.4 scanline.py

A binarizált képből vízszintes scanline-okat állít elő, majd ezekből fekete-fehér run sorozatokat készít. A run-length reprezentáció azért hasznos, mert az EAN-13 számjegyek valójában fekete és fehér sávok relatív szélességeiből állnak.

4.5 pattern_decoder.py

A run-length alapú EAN-13 dekóder a bal oldali számjegyeket L/G, a jobb oldali számjegyeket R mintákkal hasonlítja össze. A dekódolás során távolságot számít a mért run-arányok és az ismert EAN-13 minták között.

4.6 decoder.py

A klasszikus 95 bites EAN-13 bitstruktúra dekódolásáért és a checksum ellenőrzésért felel [2]. Ez a modul tartalmazza az ellenőrző számjegy számítását is.

4.7 ean13_patterns.py

Az EAN-13 L, G és R bitmintáit, valamint az első számjegyhez tartozó paritásmintákat tartalmazza [2].

4.8 barcode_reader.py

A teljes olvasási folyamatot koordináló modul. Meghívja a ROI detektort, előállítja a bináris variánsokat, scanline-okat mintavételez, megpróbálja a run-alapú dekódolást, szükség esetén bit-alapú fallback dekódolást használ, majd több találat alapján kiválasztja vagy elutasítja a végső eredményt.

4.9 stats_report.py

A statisztikai kiértékelést végző modul [9]. Nem módosítja a feldolgozási logikát, hanem a meglévő olvasófüggvényt hívja meg, majd az eredményeket státusz, hibakategória, score, találatszám és módszerek szerint összesíti.

5 Feldolgozási folyamat

A feldolgozás fő lépései a saját implementáció alapján a következők [8]:

1. Kép betöltése
2. Kép több forgatási variánsának előállítása
3. Vonalkód-gyanús ROI-k keresése
4. ROI-k előfeldolgozása szürkeárnyaltos és bináris képpé
5. Több binarizálási módszer kipróbálása
6. Több scanline mintavételezése a ROI középső tartományából
7. Run-length sorozatok előállítása
8. Run-alapú EAN-13 dekódolás

9. Sikertelenség esetén bit-alapú fallback dekódolás
10. Checksum ellenőrzés
11. Több scanline-találat összesítése
12. Bizonytalansági szabályok alkalmazása
13. OK, ROSSZ, HIBA státusz megállapítása

6 Előfeldolgozás és ROI detektálás

A ROI detektálás célja az, hogy a teljes képből a vonalkódot tartalmazó részeket kivágja [8]. A program Sobel-alapú vízszintes gradienskeresést használ, mert a vonalkód sok, egymáshoz közeli függőleges sávból áll, amelyek erős intenzitásváltozást okoznak vízszintes irányban [4], [8].

A gradiens képen Gaussian blur csökkenti a zajt, majd Otsu-küszöbölés választja szét az erősebb éleket a háttértől [5], [8]. Ezután morfológiai zárás kapcsolja össze a vonalkód egymáshoz közeli sávjait egy nagyobb blokkszerű területté [6], [8]. A kontúrokból forgatott téglalapok készülnek, majd a program méret, oldalarány és terület alapján választja ki a legerősebb jelölteket.

7 Scanline-alapú dekódolás

A program nem egyetlen sort próbál olvasni, hanem sok scanline-t mintavételez a ROI középső tartományából [8]. Ennek az oka, hogy egy termékfotón a vonalkód gyakran ferde, görbült, részben homályos vagy lokálisan sérült. Több scanline használatával nő az esély arra, hogy legalább néhány sor stabilan átmenjen a vonalkód teljes szélességén.

A scanline-okból run-length sorozat készül. Egy run azt jelenti, hogy egymás után hány azonos színű pixel következik. Az EAN-13 számjegyei sávszélesség-arányokkal írhatóak le, ezért a run-alapú dekódolás közvetlenül illeszkedik a vonalkód természetéhez.

8 Run-alapú és bit-alapú dekódolás

Az elsődleges dekódolási módszer a run-alapú mintaillesztés [8]. A program a mért run-arányokat összehasonlítja az EAN-13 ismert L, G és R számjegymintáival. Minden számjegyre egy távolságérték tartozik, minél kisebb ez az érték, annál jobban hasonlít a mért minta az adott számjegyre.

Ha a run-alapú módszer sikertelen, a program fallback módszerként megpróbálja a run sorozatot 95 bites EAN struktúrává alakítani [2], [8]. Ezután ellenőrzi a start, middle és end guard biteket, majd a 7 bites számjegycsoportokat dekódolja. Ez a fallback bizonyos esetekben segít, de általában kevésbé stabil, mint a run-alapú illesztés.

9 Hibakezelés és bizonytalansági logika

A rendszer nem minden checksum-helyes kódot fogad el automatikusan [8]. Ez fontos döntés, mert a véletlenül talált hamis minták is eredményezhetnek checksum-helyes EAN-13 számot. A program ezért score, találatszám, módszercsalád és alternatív jelöltek alapján dönt.

A bizonytalansági logika célja, hogy a scanner inkább hibát jelezzon, mintsem rossz kódot adjon vissza. Ez különösen fontos olyan környezetben, ahol a visszaadott vonalkód alapján termékazonosítás, készletkezelés vagy további automatikus feldolgozás történik.

10 Statisztika értékelése

10.1 Hibakategóriák elemzése

10.2 Score és találatszám összehasonlítása

10.3 A hibás olvasások jelentősége

10.4 Módszercsaládok szerepe

10.5 ROI detektálás értékelése

11 Eredmények értelmezése

12 Fejlesztési lehetőségek

13 Összegzés

14 Források

[1] GS1, „Barcodes,” GS1. [Online]. Elérhető: <https://www.gs1.org/standards/barcodes>. [Hozzáférés: 2026. május 14.]

[2] GS1, „EAN/UPC barcodes,” GS1. [Online]. Elérhető: <https://www.gs1.org/standards/barcodes/ean-upc>. [Hozzáférés: 2026. május 14.]

[3] GS1 US, „What is a GTIN?,” GS1 US. [Online]. Elérhető: <https://www.gs1us.org/upcs-barcodes-prefixes/what-is-a-gtin>. [Hozzáférés: 2026. május 14.]

[4] OpenCV, „Image Processing (imgproc module),” OpenCV Documentation. [Online]. Elérhető: https://docs.opencv.org/3.4/d7/da8/tutorial_table_of_content_imgproc.html. [Hozzáférés: 2026. május 14.]

[5] OpenCV-Python Tutorials, „Image Thresholding,” OpenCV-Python Tutorials. [Online]. Elérhető: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_thresholding/py_thresholding.html. [Hozzáférés: 2026. május 14.]

[6] OpenCV-Python Tutorials, „Morphological Transformations,” OpenCV-Python Tutorials. [Online]. Elérhető: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_morphological_ops/py_morphological_ops.html. [Hozzáférés: 2026. május 14.]

[7] S0dium, „DEAL KAIST Lab Barcode Main,” Kaggle. [Online]. Elérhető: <https://www.kaggle.com/datasets/s0dium/deal-kaist-lab-barcode-main/data>. [Hozzáférés: 2026. május 14.]

[8] „EAN-13 vonalkódolvasó Python implementáció,” saját fejlesztésű forráskód, 2026.

[9] „EAN-13 vonalkódolvasó statisztikai kiértékelése 1200 képen,” saját mérési eredmények, 2026.